ARTIFACT_VALIDATION

Helix OTA — Artifact Validation Pipeline

Overview

This flowchart details the **multi-stage artifact validation pipeline** that every OTA update artifact must pass through before it can be deployed to devices. The pipeline enforces structural integrity, cryptographic authenticity, and device compatibility — ensuring that only verified, correctly-targeted artifacts reach the device fleet.

Diagram

```
flowchart TB
    START(["📩 Artifact Uploaded<br/>(via Dashboard or API)"]) --> META["📋 Step 1: Metadata Extraction<br/>————————————————————<br/>• Parse manifest.json<br/>• Extract target_hardware<br/>• Extract target_build_range<br/>• Extract version / build number<br/>• Extract changelog"]

    META --> STRUCT["🔍 Step 2: Structure Validation<br/>————————————————————<br/>• ZIP structure intact?<br/>• Required files present?<br/>  - payload.bin<br/>  - payload_properties.txt<br/>  - manifest.json<br/>  - care_map.pb<br/>• File sizes within limits?<br/>• No unexpected symlinks?"]

    STRUCT --> STRUCT_OK{Structure<br/>Valid?}
    STRUCT_OK -- "✓ Pass" --> HASH["🔐 Step 3: Hash Verification<br/>————————————————————<br/>• Compute SHA-256 over<br/>  entire artifact<br/>• Compare with expected hash<br/>(provided at upload)<br/>• Compare chunk hashes<br/>(payload_properties.txt)"]
    STRUCT_OK -- "✗ Fail" --> STRUCT_FAIL["❌ Structure Check Failed<br/>————————————————————<br/>Error: INVALID_STRUCTURE<br/>Details: missing files,<br/>corrupt ZIP, size mismatch"]

    HASH --> HASH_OK{Hash<br/>Matches?}
```

```
    HASH_OK -- "✓ Pass" --> SIG["🔏 Step 4: Signature
Verification<br/>————————————————<br/>• Verify Ed25519
signature<br/>  using trusted public key<br/>• Signature
covers:<br/>  - SHA-256 hash of payload<br/>  - manifest.json<br/
>• Check signature not expired<br/>• Verify signing key not
revoked"]
    HASH_OK -- "✗ Fail" --> HASH_FAIL["❌ Hash Check Failed<br/
>————————————————<br/>Error: HASH_MISMATCH<br/>Details:
expected vs actual<br/>SHA-256 values"]

    SIG --> SIG_OK{Signature<br/>Valid?}
    SIG_OK -- "✓ Pass" --> COMPAT["🎯 Step 5: Compatibility
Check<br/>————————————————<br/>• target_hardware matches<br/>
a known device profile?<br/>• Source build range valid?<br/>  (can
update from X → Y)<br/>• No circular dependency?<br/>• Delta patch
source exists?<br/>  (for delta artifacts)<br/>• Android API level
compatible?<br/>• OEM version constraints met?"]
    SIG_OK -- "✗ Fail" --> SIG_FAIL["❌ Signature Check Failed<br/
>————————————————<br/>Error: INVALID_SIGNATURE<br/>Details: key
mismatch,<br/>signature expired,<br/>key revoked"]

    COMPAT --> COMPAT_OK{Compatibility<br/>Valid?}
    COMPAT_OK -- "✓ Pass" --> QUARANTINE["🗄️ Step 6: Quarantine
Scan<br/>————————————————<br/>• Virus/malware scan<br/>
(ClamAV integration)<br/>• Check for known<br/>  suspicious
patterns<br/>• Verify no priv-esc<br/>  in post-install scripts"]
    COMPAT_OK -- "✗ Fail" --> COMPAT_FAIL["❌ Compatibility Check
Failed<br/>————————————————<br/>Error: INCOMPATIBLE<br/
>Details: unknown hardware,<br/>invalid source range,<br/>missing
delta source"]

    QUARANTINE --> QUAR_OK{Scan<br/>Clean?}
    QUAR_OK -- "✓ Clean" --> READY(["✅ Artifact READY<br/
>————————————————<br/>Status: READY_FOR_DEPLOY<br/>Stored in
MinIO<br/>Metadata in PostgreSQL<br/>Available for campaigns"])
    QUAR_OK -- "✗ Threat" --> QUAR_FAIL["❌ Quarantine Scan
Failed<br/>————————————————<br/>Error: SECURITY_THREAT<br/
>Details: malware detected,<br/>suspicious pattern,<br/>dangerous
script"]

    STRUCT_FAIL --> REJECT["🚫 Artifact REJECTED<br/
>————————————————<br/>Status: REJECTED<br/>Stored in quarantine
zone<br/>Operator notified<br/>Audit log entry created"]
    HASH_FAIL --> REJECT
    SIG_FAIL --> REJECT
    COMPAT_FAIL --> REJECT
    QUAR_FAIL --> REJECT

    REJECT --> NOTIFY["🔔 Notification<br/>————————————————<br/
>• Dashboard alert<br/>• Email to uploaders<br/>• Webhook (if
configured)<br/>• Audit log entry"]

    style START fill:#4FC3F7,stroke:#0277BD,color:#000
    style READY fill:#66BB6A,stroke:#2E7D32,color:#000
```

```
style REJECT fill:#EF5350,stroke:#C62828,color:#fff
style NOTIFY fill:#FFA726,stroke:#E65100,color:#000
style META fill:#E3F2FD,stroke:#1565C0,color:#000
style STRUCT fill:#E3F2FD,stroke:#1565C0,color:#000
style HASH fill:#FFF3E0,stroke:#E65100,color:#000
style SIG fill:#FFF3E0,stroke:#E65100,color:#000
style COMPAT fill:#E8F5E9,stroke:#2E7D32,color:#000
style QUARANTINE fill:#FCE4EC,stroke:#C62828,color:#000
style STRUCT_OK fill:#BBDEFB,stroke:#1565C0,color:#000
style HASH_OK fill:#FFE0B2,stroke:#E65100,color:#000
style SIG_OK fill:#FFE0B2,stroke:#E65100,color:#000
style COMPAT_OK fill:#C8E6C9,stroke:#2E7D32,color:#000
style QUAR_OK fill:#F8BBD0,stroke:#C62828,color:#000
```

Validation Pipeline Stages

Step	Check	Pass Criteria	Failure Action
1. Metadata Extraction	Parse manifest.json	All required fields present	—
2. Structure Validation	ZIP integrity, required files	Valid ZIP, all files present, sizes within limits	REJECT — INVALID_STRUCTURE
3. Hash Verification	SHA-256 of entire artifact	Computed hash matches declared hash	REJECT — HASH_MISMATCH
4. Signature Verification	Ed25519 signature over hash + manifest	Valid signature from trusted key, not expired, key not revoked	REJECT — INVALID_SIGNATURE
5. Compatibility Check	Hardware, build range, dependencies	Target HW known, source range valid, no circular deps	REJECT — INCOMPATIBLE
6. Quarantine Scan	Malware scan, pattern check	Clean scan result	REJECT — SECURITY_THREAT

Key Design Decisions

1. **Order matters**: Checks run cheapest → most expensive. Structure validation is fast; signature verification and quarantine scans are slower.
2. **Fail-fast**: The pipeline stops at the first failure — no point checking signatures on a structurally corrupt file.
3. **Quarantine zone**: Rejected artifacts are moved to a separate MinIO bucket (quarantine zone) for forensic analysis, not deleted.
4. **Re-validation**: If the trusted key set changes (key rotation), all READY artifacts are re-validated against the new keys.

5. **Delta artifacts**: Delta (incremental) patches require an additional check that the source artifact exists and is itself in READY state.